Data Mining and Machine Learning

A Lakehouse Approach

Data Mining Course

2026

Contents

Chapter 1

Introduction

1.1 Environment Setup

To get started with this course, you need to install the required dependencies. The project uses Python and PySpark for data processing and machine learning.

1.1.1 Installation

Install the dependencies using one of the following methods:

```
# Install dependencies
pip install -e .
```

Listing 1.1: Installation via pip

Alternatively, you can use the provided Makefile:

```
# Setup using Makefile
make setup
```

Listing 1.2: Installation via Makefile

1.1.2 Verification

After installation, verify that everything is working correctly by running:

```
# Verify installation
python -m md_lakehouse --help
```

1.2 Team Configuration

Each team has a unique configuration that generates different but comparable datasets. This ensures that while each team works with distinct data, the analytical challenges and learning objectives remain consistent across all teams.

1.2.1 Configuration File

Your team's configuration is stored in: `configs/teams/team0X.yaml`

Team Configuration

Make sure to use your team's specific configuration file. Each configuration has a unique seed that ensures reproducibility while maintaining data diversity across teams.

1.2.2 Key Parameters

The configuration file contains several important parameters:

- **team_id**: Unique identifier for your team
- **seed**: Random seed for reproducibility of data generation
- **population_size**: Number of users in the generated dataset
- **churn_base_rate**: Base rate of customer churn
- **lifetime_months**: Simulation period in months
- **payment_methods**: Available payment methods
- **subscription_plans**: Different subscription tiers

These parameters control various aspects of the data generation process, allowing you to create realistic business scenarios for analysis.

1.3 Project Structure

Understanding the project structure is crucial for navigating the codebase and completing assignments. The project follows a modular architecture that separates concerns and facilitates testing and maintenance.

1.3.1 Directory Layout

```
.
|-- configs/              # Configuration files
|   |-- base.yaml
|   `-- teams/
|       |-- team01.yaml
|       |-- team02.yaml
|       `-- team03.yaml
|-- src/md_lakehouse/     # Source code
|   |-- generator/        # Data generation
|   |-- pipelines/        # ETL pipelines
|   |-- features/         # Feature engineering
|   |-- models/           # Machine learning models
|   `-- evaluation/       # Model evaluation
|-- tests/                # Test suites
|-- notebooks/            # Jupyter notebooks
|-- data/                 # Generated data (git-ignored)
`-- docs/                 # Documentation
```

Listing 1.3: Project Directory Structure

1.3.2 Key Modules

Generator Module

The `generator/` module contains the data generation logic. It creates synthetic but realistic datasets that simulate a subscription-based business with users, subscriptions, payments, usage logs, and business events.

- `generate.py`: Main data generation orchestrator
- `logic.py`: Business logic for churn, LTV, and behavior simulation
- `schemas.py`: Data schemas using PySpark StructTypes

Pipelines Module

The `pipelines/` module implements ETL (Extract, Transform, Load) pipelines following the Lakehouse architecture:

- `bronze_to_silver.py`: Data cleaning and validation
- `silver_to_gold.py`: Feature engineering and aggregations
- `quality.py`: Data quality validation rules

Models Module

The `models/` module contains machine learning model implementations:

- `churn.py`: Customer churn prediction models
- `ltv.py`: Customer Lifetime Value prediction
- `segmentation.py`: Customer segmentation clustering

Evaluation Module

The `evaluation/` module provides tools for model assessment:

- `metrics.py`: Performance metrics calculation
- `reports.py`: Report generation and visualization

1.4 Getting Started

Now that you understand the environment setup and project structure, you're ready to begin the hands-on work.

1.4.1 Next Steps

Follow these steps to get started with the course:

1. **Explore** the notebook `01_kdd.ipynb` to understand the KDD process and how it maps to the Lakehouse architecture
2. **Generate** data for your team using the provided generators
3. **Implement** the ETL pipelines to transform raw data into analytical datasets
4. **Train** machine learning models for churn prediction, LTV estimation, and customer segmentation
5. **Evaluate** your results and compare them with other teams

1.4.2 Learning Objectives

By the end of this course, you will be able to:

- Understand the complete data mining lifecycle using the KDD process
- Design and implement scalable data pipelines using the Lakehouse architecture
- Apply data quality principles to ensure reliable analytics
- Build and evaluate machine learning models for real-world business problems
- Use modern tools like PySpark, Parquet, and versioned data storage
- Work collaboratively in teams to solve complex data challenges

Important

All code should be reproducible. Using the same configuration and seed should always produce identical results. This is crucial for collaboration, debugging, and grading.

1.5 Course Philosophy

This course takes a practical, hands-on approach to data mining. Rather than just learning theory, you'll work with a complete end-to-end project that simulates real business scenarios.

1.5.1 Why the Lakehouse Architecture?

Traditional data architectures force you to choose between data lakes (flexible but unstructured) and data warehouses (structured but rigid). The Lakehouse architecture combines the best of both worlds:

- **Open formats**: Use Parquet for efficient columnar storage
- **Versioning**: Track changes over time with dated snapshots
- **Schema enforcement**: Validate data quality at each layer
- **Separation of concerns**: Bronze, Silver, and Gold layers with clear purposes
- **Scalability**: Process large datasets with distributed computing

1.5.2 Why Synthetic Data?

Using generated data provides several advantages for learning:

- **No privacy concerns**: Work freely without sensitive information
- **Controlled complexity**: Start simple, add features progressively
- **Known ground truth**: Validate models against known patterns
- **Team diversity**: Each team has unique data while maintaining comparability
- **Reproducibility**: Same seed produces identical datasets

Let's begin this journey into data mining and machine learning!

Chapter 2

The KDD Process and Lakehouse Architecture

This chapter introduces the Knowledge Discovery in Databases (KDD) process and shows how it maps to the modern Lakehouse architecture. You'll learn how each stage of KDD corresponds to specific layers in our data pipeline.

2.1 Mapping KDD to Lakehouse

The KDD process provides a systematic approach to extracting knowledge from data. In this course, we map each KDD stage to a corresponding layer in the Lakehouse architecture, creating a clear and reproducible workflow.

2.1.1 The Mapping

Table **??** shows how KDD stages correspond to Lakehouse layers:

Table 2.1: KDD Process mapped to Lakehouse Architecture

KDD Stage	**Lakehouse Layer**	**Description**
Selection	**Bronze**	Raw data as selected from sources
Preprocessing	**Silver**	Cleaned and validated data
Transformation	**Gold**	Features and aggregations
Data Mining	**Models**	Trained machine learning models
Evaluation	**Reports**	Metrics and model evaluation

2.1.2 Why This Mapping?

This mapping provides several benefits:

- **Clear separation of concerns**: Each layer has a specific purpose
- **Incremental refinement**: Data quality improves through the pipeline
- **Reproducibility**: Each layer can be regenerated from previous layers
- **Testing**: Quality gates at each transition point
- **Debugging**: Easy to identify where issues occur in the pipeline

2.2 Lakehouse Architecture

The Lakehouse architecture combines the flexibility of data lakes with the structure and performance of data warehouses. Let's explore why this matters and how it works.

2.2.1 Why Lakehouse?

Traditional data architectures force trade-offs:

- **Data Lakes**: Flexible and low-cost, but lack structure and data quality guarantees
- **Data Warehouses**: Structured and performant, but rigid and expensive

The Lakehouse architecture provides:

- **Flexibility**: Store any type of data in open formats
- **Low cost**: Use object storage (e.g., S3, Azure Blob)
- **Structure**: Schema enforcement and validation
- **Performance**: Columnar formats optimized for analytics
- **Versioning**: Time-travel capabilities with snapshots

2.2.2 Key Principles

The Lakehouse architecture follows four key principles:

1. **Open storage formats**: Use Parquet for efficient, columnar, compressed storage
2. **Versioning**: Maintain snapshots with partitions by `run_date`
3. **Semantic layers**: Bronze → Silver → Gold progression
4. **Data quality**: Validation and quality gates at each layer

Parquet Format

Parquet is a columnar storage format that provides:

- Efficient compression (typically 75-90% reduction)
- Column pruning (read only needed columns)
- Predicate pushdown (filter at storage level)
- Schema evolution support
- Cross-platform compatibility

Time-Travel with Snapshots

Each table is partitioned by `run_date`, creating immutable snapshots:

```
data/bronze/users/
|-- run_date=2026-01-01/
|   |-- part-00000.parquet
|   `-- part-00001.parquet
|-- run_date=2026-02-01/
|   |-- part-00000.parquet
|   `-- part-00001.parquet
`-- run_date=2026-03-01/
    |-- part-00000.parquet
    `-- part-00001.parquet
```

Listing 2.1: Snapshot directory structure

2.3 Bronze Layer: Raw Data

The Bronze layer stores data exactly as it's generated or received from source systems. No transformations are applied at this stage.

2.3.1 Purpose

The Bronze layer serves as the single source of truth:

- Immutable record of all incoming data
- Historical archive for auditing and debugging
- Starting point for all downstream pipelines
- Recovery point if issues occur in later stages

2.3.2 Bronze Tables

Our data generator creates five Bronze tables:

`users` Customer demographic and signup information

`subscriptions` Subscription lifecycle (start, changes, cancellations)

`payments` Payment transactions with amounts and methods

`usage_logs` Daily usage metrics per user

`events` Business events (promotions, outages, policy changes)

2.3.3 Data Format

Bronze data is stored in Parquet format with the following characteristics:

- Partitioned by `run_date=YYYY-MM-DD`
- Original data types preserved
- No cleaning or validation applied
- Minimal indexing (only partition columns)

Bronze Best Practices

Never modify Bronze data after it's written. If source data has errors, they should be visible in Bronze and corrected in Silver. This maintains a true audit trail.

2.4 Silver Layer: Validated Data

The Silver layer transforms Bronze data into clean, validated, analysis-ready datasets. This is where data quality is enforced.

2.4.1 Purpose

The Silver layer ensures data quality:

- Remove duplicates and invalid records
- Validate data ranges and constraints
- Enforce referential integrity
- Normalize timestamps and formats
- Apply correct data types

2.4.2 Transformations

Key transformations applied in the Bronze → Silver pipeline:

1. **Deduplication**: Remove exact duplicate records
2. **Range validation**: Ensure values are within valid ranges
   - Age: 14-90 years
   - Payment amounts: > 0
   - Dates: Within simulation period
3. **Referential integrity**: Validate foreign key relationships
   - All `subscription.user_id` exist in `users`
   - All `payment.user_id` exist in `users`

- All `usage_log.user_id` exist in `users`

4. **Timestamp normalization**: Convert all timestamps to UTC
5. **Type casting**: Ensure correct Spark data types

2.4.3 Data Quality Gates

Quality rules are enforced using a validation framework:

Definition 2.1 (Quality Rule)**.** A quality rule is a boolean condition that data must satisfy to pass validation. Rules can be:

- **Blocking**: Reject records that fail (e.g., negative amounts)
- **Warning**: Log failures but allow records (e.g., unusual patterns)

Common quality checks include:

- **Not null**: Critical fields must be present
- **Unique**: Keys must be unique within a snapshot
- **Range**: Values must be within valid ranges
- **Format**: Strings must match expected patterns (e.g., emails)
- **Referential**: Foreign keys must reference existing records

```python
from pyspark.sql import functions as F

def validate_age_range(df):
    """Validate user ages are between 14 and 90."""
    return df.filter(
        (F.col("age") >= 14) & (F.col("age") <= 90)
    )

def validate_positive_amounts(df):
    """Ensure payment amounts are positive."""
    return df.filter(F.col("amount") > 0)
```

Listing 2.2: Example quality rule implementation

2.4.4 Rejected Records

Records that fail validation are:

1. Logged to a rejection table with reason codes
2. Monitored for patterns indicating upstream issues
3. Reported in data quality metrics

This allows teams to identify and fix data quality problems at their source.

2.5 Gold Layer: Analytical Data

The Gold layer contains feature-engineered, aggregated data optimized for analytics and machine learning. This is where business logic is applied to create insights.

2.5.1 Purpose

The Gold layer serves analysts and data scientists:

- Pre-computed features for machine learning
- Aggregated metrics for reporting and dashboards
- Business KPIs calculated consistently
- Optimized for query performance

2.5.2 Gold Tables

Our pipeline creates several Gold tables:

`fact_usage_daily` Daily usage metrics aggregated by user

`fact_revenue_daily` Daily revenue metrics and MRR calculations

`churn_features` Features for churn prediction models

`churn_labels` Historical churn labels for training

`kpis_daily` Business KPIs tracked over time

`customer_segments` Customer segmentation results

2.5.3 Feature Engineering

Features are computed using temporal windows and aggregations:

Window Functions

Temporal features look at behavior over different time periods:

- **7-day window**: Recent behavior (last week)
- **30-day window**: Monthly patterns
- **90-day window**: Long-term trends

```python
from pyspark.sql import Window
from pyspark.sql import functions as F

# Define 30-day window
window_30d = Window.partitionBy("user_id").orderBy("date").rowsBetween
    (-29, 0)

```

```python
# Compute rolling metrics
df_features = df.withColumn(
    "usage_30d_avg",
    F.avg("daily_usage").over(window_30d)
).withColumn(
    "payment_30d_sum",
    F.sum("payment_amount").over(window_30d)
)
```

Listing 2.3: Computing window features

Aggregation Types

Common aggregations include:

- **Count**: Number of events (logins, payments)
- **Sum**: Total amounts (revenue, usage)
- **Average**: Mean values (session duration)
- **Min/Max**: Extremes (first/last activity)
- **Standard deviation**: Variability measures

Ratios and Indicators

Derived metrics often provide more signal than raw counts:

- Usage ratio: $\frac{\text{actual usage}}{\text{plan limit}}$
- Payment consistency: $\frac{\text{successful payments}}{\text{total attempts}}$
- Engagement score: Composite metric combining multiple indicators

2.5.4 Business KPIs

Key Performance Indicators are standardized metrics tracked over time:

MRR Monthly Recurring Revenue: Sum of active subscription values

Churn Rate Percentage of customers who cancel: $\frac{\text{churned}}{\text{active at start}}$

ARPU Average Revenue Per User: $\frac{\text{total revenue}}{\text{active users}}$

LTV Lifetime Value: Expected total revenue from a customer

Exercise: Calculate Churn Rate

Given:

- 1,000 customers at the start of January
- 50 customers cancelled during January

- 75 new customers acquired in January

Calculate the January churn rate.
Solution: Churn rate $= \frac{50}{1000} = 5\%$
Note: New customers don't affect the churn rate calculation for this period.

2.6 Time-Travel with Snapshots

One of the most powerful features of the Lakehouse architecture is the ability to "time-travel" through data versions.

2.6.1 Snapshot Mechanics

Each table is partitioned by `run_date`, creating immutable snapshots:

```python
from md_lakehouse.common.io import read_parquet_snapshot

# Read specific snapshot
df_jan = read_parquet_snapshot(
    spark,
    path="data/silver",
    table="users",
    run_date="2026-01-01"
)

# Read latest snapshot
df_latest = read_latest_snapshot(
    spark,
    path="data/silver",
    table="users"
)
```

Listing 2.4: Reading specific snapshots

2.6.2 Benefits of Time-Travel

1. **Reproducibility**: Re-run analysis with historical data
2. **Auditing**: Track how data changed over time
3. **Debugging**: Identify when issues were introduced
4. **Rollback**: Revert to previous versions if needed
5. **Comparison**: Analyze temporal evolution
6. **Testing**: Use specific snapshots for test validation

2.6.3 Snapshot Strategy

When should you create a new snapshot?

- **Scheduled runs**: Daily, weekly, or monthly refreshes
- **Data updates**: When source data changes significantly
- **Pipeline changes**: After modifying transformation logic
- **Milestones**: Before major releases or presentations

Storage Considerations

Snapshots consume storage space. Balance between:

- Granularity: How many snapshots to keep
- Retention: How long to keep old snapshots
- Cost: Storage costs vs. value of history

Consider implementing a retention policy (e.g., keep daily for 30 days, weekly for 1 year, monthly forever).

2.7 Data Quality Framework

Data quality is not optional—it's fundamental to reliable analytics and machine learning. Our framework enforces quality at every layer.

2.7.1 Quality Dimensions

We validate data across multiple dimensions:

Completeness Are all required fields present? (Not null checks)

Validity Are values within acceptable ranges? (Range checks)

Uniqueness Are keys truly unique? (Duplicate detection)

Consistency Do relationships hold? (Referential integrity)

Timeliness Is data current? (Freshness checks)

Accuracy Does data represent reality? (Business rule validation)

2.7.2 Implementation Pattern

Quality rules follow a consistent pattern:

```python
from dataclasses import dataclass
from typing import Callable
from pyspark.sql import DataFrame

@dataclass
class QualityRule:
    name: str
    description: str
    check: Callable[[DataFrame], DataFrame]
    severity: str  # "error" or "warning"

def apply_quality_rules(df: DataFrame, rules: list[QualityRule]) ->
    DataFrame:
    """Apply a list of quality rules to a DataFrame."""
    valid_df = df

    for rule in rules:
        before_count = valid_df.count()
        valid_df = rule.check(valid_df)
        after_count = valid_df.count()

        rejected = before_count - after_count
        if rejected > 0:
            print(f"{rule.name}: Rejected {rejected} records")

    return valid_df
```

Listing 2.5: Quality rule pattern

2.8 Reflection and Next Steps

Let's consolidate what we've learned and look ahead.

2.8.1 Discussion Questions

Consider these questions:

1. **Why separate layers?** What would happen if we combined Bronze, Silver, and Gold into one table?
2. **Snapshot value**: How do snapshots help with reproducibility? Can you think of a scenario where they'd be critical?
3. **Quality gates**: Why validate in Silver rather than Bronze? What are the trade-offs?
4. **KDD vs ETL**: How does the KDD process differ from traditional ETL? What additional value does it provide?

2.8.2 Practical Exercise

Before the next session, complete these tasks:

1. **Generate Bronze data** for your team using the data generator
2. **Explore tables** using PySpark to understand the schema and data
3. **Verify reproducibility** by generating data twice with the same seed and comparing results
4. **Read the Silver pipeline** code to understand the transformation logic

```
# Generate Bronze data
python -m md_lakehouse generate --config configs/teams/team01.yaml

# Verify the data was created
ls -lh data/bronze/
```

Listing 2.6: Generating data for your team

2.8.3 Key Takeaways

Remember these core concepts:

- KDD maps cleanly to Lakehouse layers (Bronze/Silver/Gold)
- Each layer has a specific purpose and quality level
- Snapshots enable reproducibility and time-travel
- Data quality must be enforced systematically
- Features are engineered in Gold for ML consumption
- Open formats (Parquet) provide flexibility and performance

You now understand the theoretical foundation. In the next chapters, we'll implement these concepts hands-on, building complete ETL pipelines and training machine learning models.

Let's get started!